

Laboratorio01(1)

October 1, 2026

1 Laboratorio de Práctica

1.1 Nivel Básico

1.1.1 Ejercicio 1: Análisis de una expresión algebraica

Una expresión matemática se recibe como una **cadena** de caracteres. Definir una función `analizar_expresion(expr)` que retorne una **tupla** (`balanceada`, `n_operadores`, `variables`) donde:

- `balanceada` es `True` si los paréntesis `( )` y corchetes `[ ]` están correctamente balanceados y anidados, y `False` en caso contrario. (Ayuda: usar una **lista como pila** y un **diccionario** que asocie cada símbolo de cierre con su símbolo de apertura.)
- `n_operadores` es la cantidad de operadores `+ - * / ^` que aparecen en la expresión.
- `variables` es una **lista ordenada** con las letras distintas que aparecen en la expresión (por ejemplo `['x', 'y']`).

Probar con las siguientes expresiones e imprimir el resultado de cada una con un mensaje del tipo:

`(x+1)*(x-1) -> balanceada: True, operadores: 3, variables: ['x']`

```
[1]: expresiones = ["(x+1)*(x-1)", "[ (2*x)+3 ]/y", "((2*x)+3", "3*x^2-(y+4))",  
↪ "[ (x+1) ]"]
```

```
[ ]: # Solución:  
simbolos = {  
    "]" : "[",  
    ")" : "(",  
}  
  
def expr(texto):  
    pila = []  
    for expression in expresiones:  
        for letra in expression:  
            pila.append(expression)  
    b = False  
    return [b]
```

```
[23]: simbolos["(x+1)*(x-1)"]
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 simbolos["(x+1)*(x-1)"]

KeyError: '(x+1)*(x-1)'

```

[]:

1.1.2 Ejercicio 2: Estadística descriptiva de una muestra

Se tiene una muestra de datos en una lista. Sin usar el módulo `statistics` para el cálculo:

- Definir `estadisticas(datos)` que retorne una **tupla** (media, mediana, varianza, desviacion), usando la **varianza muestral** (dividiendo entre $n - 1$). La mediana debe calcularse **sin modificar** la lista original.
- Con una **lista por comprensión**, crear `z` con los puntajes estandarizados $z_i = (x_i - \bar{x})/s$, redondeados a 4 decimales.
- Crear la lista `atipicos` con los datos cuyo $|z_i| > 2$ (puede usar `zip`).
- Verificar los resultados comparando con `statistics.mean`, `statistics.median` y `statistics.variance` usando `math.isclose`.

Imprimir los resultados y comprobar que `datos` conserva su orden original.

```

[ ]: import math
import statistics

datos = [12, 15, 14, 10, 13, 16, 11, 14, 45, 13]

```

```

[97]: # Solución:
def estadisticas(datos):
    # Media
    media = sum(datos)/len(datos)

    # Mediana
    if len(datos)%2==0:
        aux1 = int(len(datos)/2)
        aux2 = int(len(datos)/2) + 1
    else:
        aux1 = aux2 = int(len(datos)/2) + 1
    centro = sorted(datos)[aux1 - 1 : aux2]
    mediana = sum(centro)/len(centro)

    # Varianza
    var = 0
    for n in datos:
        var += (n - media)**2

```

```

    var = var / (len(datos)-1)

    return [media, mediana, var, math.sqrt(var)]

media, mediana, var, s = estadisticas(datos)
datos_centalizados = [round((x - media)/s,4) for x in datos]
atipicos = [z for z in datos_centalizados if abs(z)>2]

if math.isclose(media,statistics.mean(datos)) and math.
    ↪isclose(mediana,statistics.median(datos)) and math.isclose(var,statistics.
    ↪variance(datos)):
    print("Comprobación realizada: Profe lo logré c:")
else:
    print("Rotundo fracaso: Profe le fallé :c")
print("\n--Resultados--")
print("Media: ", media)
print("Mediana: ", mediana)
print("Varianza: ", var)
print("Desviancion: ", s)
print("Valores centralizados:", datos_centalizados)
print("Valores atipicos:", atipicos)

# datos conserva su orden original
print("\nDatos = ", datos)

```

Comprobación realizada: Profe lo logré c:

--Resultados--

Media: 16.3

Mediana: 13.5

Varianza: 104.9

Desviancion: 10.242070103255495

Valores centralizados: [-0.4198, -0.1269, -0.2246, -0.6151, -0.3222, -0.0293, -0.5175, -0.2246, 2.8022, -0.3222]

Valores atipicos: [2.8022]

Datos = [12, 15, 14, 10, 13, 16, 11, 14, 45, 13]

1.1.3 Ejercicio 3: Números complejos como tuplas

Un número complejo $z = a + bi$ se representa como la **tupla** (a, b). Sin usar el tipo `complex` de Python, definir:

- `producto(z, w)` que retorne el producto de dos complejos como tupla. (Desempaquetar: a, b = z.)
- `modulo(z)` que retorne $|z|$.
- `a_cadena(z)` que retorne el número como **cadena** con el formato "5+5i" o "3-2i" (el signo de la parte imaginaria debe escribirse correctamente).

- `potencia(z, n)` que retorne z^n (con $n \geq 0$ entero) multiplicando repetidamente con producto.

Pruebas:

1. Imprimir $(1 + 2i)(3 - i)$ en formato cadena.
2. Calcular $(1 + i)^8$ e i^4 , e imprimirlos. Verificar con `math.isclose` que $|(1 + i)^8| = |1 + i|^8$.
3. Dada la lista `zs`, imprimir cada número en formato cadena junto con su módulo, **ordenados por módulo creciente**.

```
[5]: import math
```

```
zs = [(1, 2), (3, -1), (0, 1), (-2, -2)]
```

```
[134]: # Solución:
```

```
def producto(z,w):
    a , b = z
    c, d = w
    return [a*c - b*d, a*d + b*c]

def modulo(z):
    a , b = z
    return math.sqrt(a**2 + b**2)

def a_cadena(z):
    a,b = z
    if b < 0:
        return f"{a} - {abs(b)}i"
    if b > 0:
        return f"{a} + {abs(b)}i"
    else:
        return f"{a}"          # b == 0

def potencia(z, n):
    res = [1,0]
    for i in range(n):
        res = producto(z, res)
    return res

# Pruebas
print("Prueba 1")
print(a_cadena(producto([1,2], [3, -1])))

print("\nPrueba 2")
print(a_cadena(potencia([1,1], 8)))
print(a_cadena(potencia([0,1], 4)))
print("Verificacion:", math.isclose(modulo(potencia([1,1], 8)),
    ↪ modulo([1,1])**8))
```

```

print("\nPrueba 3")
zs.sort(key=modulo)
print("Numero \t\t Modulo")
for num in zs:
    print(f"{a_cadena(num)}\t\t{modulo(num)}")

```

Prueba 1

5 + 5i

Prueba 2

16

1

Verificacion: True

Prueba 3

Numero	Modulo
0 + 1i	1.0
1 + 2i	2.23606797749979
-2 - 2i	2.8284271247461903
3 - 1i	3.1622776601683795

1.1.4 Ejercicio 4: Polinomios como diccionarios

Un polinomio se puede representar de forma compacta con un **diccionario** {exponente: coeficiente} que guarda solo los términos no nulos. Por ejemplo, $p(x) = 3x^5 - 2x^2 + 7$ es {5: 3, 2: -2, 0: 7}.

Definir las siguientes funciones:

- `evaluar(p, x)` — evalúa el polinomio en x .
- `derivar(p)` — retorna un **nuevo diccionario** con la derivada de p (los términos constantes desaparecen).
- `sumar(p, q)` — retorna un nuevo diccionario con $p + q$, **eliminando** los términos cuyo coeficiente resulte 0. No modificar p ni q .
- `a_cadena(p)` — retorna el polinomio como **cadena**, con los términos en orden decreciente de exponente. Reglas: el exponente 1 se escribe x , el exponente 0 solo el coeficiente, y el coeficiente siempre se escribe (aunque sea 1). Ejemplo: {4: 15, 1: -4} $\rightarrow$ "15x⁴ - 4x". El polinomio vacío es "0".

Pruebas con $p = \{5: 3, 2: -2, 0: 7\}$ y $q = \{5: -3, 1: 4, 0: -7\}$:

1. Imprimir p , p' y $p + q$ en formato cadena.
2. Evaluar $p(2)$ y $p'(1)$.
3. Verificar numéricamente que $(p + q)(3) = p(3) + q(3)$.

```

[7]: p = {5: 3, 2: -2, 0: 7}
     q = {5: -3, 1: 4, 0: -7}

```

```
[179]: # Solución:
def evaluar(p,x):
    potencias = list(p.keys())
    coeficientes = list(p.values())
    return sum([coeficientes[i]*x**potencias[i] for i in range(len(p))])

def derivar(p):
    potencias = [n - 1 for n in list(p.keys()) if n-1>=0 ]
    coeficientes = [list(p.values())[i] * list(p.keys())[i] for i in
↪range(len(potencias))]
    return dict(zip(potencias, coeficientes))

def a_cadena(p):
    texto = ""
    for i in range(len(p)):
        coeficiente = list(p.values())[i]
        if(coeficiente >0):
            texto += f" + {coeficiente}"
        else:
            texto += f" {coeficiente}"
        potencia = list(p.keys())[i]
        if potencia != 0:
            texto += "x"
            if potencia !=1:
                texto += f"^{list(p.keys())[i]}"
    return texto

# Pruebas
print(a_cadena(p))
print(a_cadena(derivar(p)))
```

```
+ 3x^5 -2x^2 + 7
+ 15x^4 -4x
```

```
[176]: derivar(p)
```

```
[176]: {4: 15, 1: -4, -1: 0}
```

1.2 Nivel Intermedio

Comprensiones de listas, tuplas como registros, diccionarios en cálculos, funciones como valores, manejo de errores, módulo `math`.

1.2.1 Ejercicio 5: Aproximaciones del número e

El número e puede aproximarse de dos maneras:

$$S_n = \sum_{k=0}^n \frac{1}{k!} \quad a_n = \left(1 + \frac{1}{n}\right)^n$$

- Definir `sumas_parciales(N)` que retorne la **lista** `[S_0, S_1, ..., S_{N-1}]`.
- Definir `primer_indice(sumas, tol)` que retorne la **tupla** `(n, S_n, error)` del primer índice cuyo error absoluto respecto a `math.e` sea menor que `tol`, o `None` si ninguno lo cumple.
- Con una **lista por comprensión** crear `tabla`, una lista de tuplas `(n, S_n, error)` para `N = 15`, e imprimir las primeras 8 filas con formato alineado.
- Con otra comprensión, crear `limite`, lista de tuplas `(n, a_n, error)` para `n` en `10, 100, 1000, ..., 10^6`, e imprimirla.
- Imprimir `primer_indice(sumas, 1e-6)` y comparar: ¿cuál de las dos aproximaciones converge más rápido?

[9]: *# Solución:*

1.2.2 Ejercicio 6: Variables aleatorias discretas

Una variable aleatoria discreta se representa con un **diccionario** `{valor: probabilidad}`.

- Definir `validar(dist)` que lance un `ValueError` con el mensaje "Probabilidad negativa" si alguna probabilidad es negativa, y con el mensaje "Las probabilidades no suman 1" si la suma no es 1 (usar `math.isclose`).
- Definir `esperanza(dist)` y `varianza(dist)` que calculen $E[X] = \sum x p(x)$ y $\text{Var}(X) = \sum (x - \mu)^2 p(x)$.
- Definir `suma_independientes(X, Y)` que retorne el diccionario de la distribución de $Z = X + Y$, donde X e Y son independientes: $P(Z = z) = \sum_{x+y=z} P(X = x)P(Y = y)$. (Ayuda: dos bucles anidados sobre `items()` y `get`.)

Pruebas:

1. Validar `X, Y` y la distribución incorrecta `malo`, manejando los errores con `try/except` e imprimiendo el mensaje.
2. Calcular `Z = suma_independientes(X, Y)`, imprimirla ordenada por valor (con 4 decimales) y validarla.
3. Verificar con `math.isclose` que $E[Z] = E[X] + E[Y]$ y que $\text{Var}(Z) = \text{Var}(X) + \text{Var}(Y)$.
4. Calcular $P(Z \geq 3)$ con una comprensión sobre el diccionario.

```
[10]: import math

X = {0: 0.2, 1: 0.5, 2: 0.3}
Y = {1: 0.4, 3: 0.6}
malo = {0: 0.5, 1: 0.6}
```

[11]: *# Solución:*

1.2.3 Ejercicio 7: Método de bisección

Para una función continua f con $f(a)f(b) < 0$, el método de bisección aproxima una raíz en $[a, b]$ calculando el punto medio $c = (a + b)/2$ y quedándose con la mitad del intervalo donde f cambia

de signo.

Definir `biseccion(f, a, b, tol=1e-6)` que:

- Lance un `ValueError` con el mensaje "f(a) y f(b) deben tener signos opuestos" si $f(a)f(b) > 0$.
- Itere hasta que $|f(c)| < \text{tol}$ o la mitad de la longitud del intervalo sea menor que `tol` (máximo 100 iteraciones).
- Retorne una **tupla** (`raiz, historial`), donde `historial` es una **lista de tuplas** (`n, a, b, c, f(c)`), una por iteración.

Se proporciona un **diccionario** `problemas` que asocia el nombre de cada ecuación con una tupla (`funcion, a, b`). Recorrerlo con `.items()` (desempaquetando en el bucle `for`), resolver cada problema dentro de un `try/except` e imprimir:

```
x^2 - 2: raíz  1.414214 (21 iteraciones)
```

o el mensaje de error si el problema no es válido. Finalmente, imprimir las **primeras 5 filas del historial** del primer problema con formato de columnas alineadas.

```
[12]: import math

problemas = {
    "x^2 - 2": (lambda x: x**2 - 2, 0, 2),
    "cos(x) - x": (lambda x: math.cos(x) - x, 0, 1),
    "x^3 + x + 1": (lambda x: x**3 + x + 1, -1, 0),
    "x^2 + 1": (lambda x: x**2 + 1, -1, 1),
}
```

```
[13]: # Solución:
```

1.3 Nivel Avanzado

Programación orientada a objetos con listas anidadas, tuplas y cadenas; validación y excepciones.

1.3.1 Ejercicio 8: Clase Matriz

Diseñar una clase `Matriz` para trabajar con matrices reales, **sin usar numpy**. La matriz se almacena como una **lista de listas** (cada sublista es una fila).

La clase debe tener:

- `__init__(self, filas)` — recibe la lista de listas y guarda una **copia** en `self.filas`. Lanza `ValueError("Matriz inválida")` si está vacía o si las filas no tienen todas la misma longitud.
- `forma(self)` — retorna una **tupla** (`num_filas, num_columnas`).
- `transpuesta(self)` — retorna un **nuevo objeto Matriz** con la transpuesta. (Ayuda: `zip(*self.filas)`.)
- `traza(self)` — retorna la suma de la diagonal. Lanza `ValueError("La traza requiere una matriz cuadrada")` si no es cuadrada.

- `__mul__(self, otro)` — producto matricial `self * otro`, retornando un nuevo objeto `Matriz`. Lanza `ValueError` con un mensaje que incluya ambas formas (por ejemplo "Dimensiones incompatibles: (2, 3) y (2, 3)") si el producto no está definido.
- `__eq__(self, otro)` — retorna `True` si ambas matrices tienen las mismas filas.
- `__str__(self)` — retorna una **cadena** con una fila por línea y cada elemento alineado a la derecha en un campo de ancho 6.

Probar con el siguiente código (la salida debe mostrar AB , su traza, y que se cumple $(AB)^T = B^T A^T$; los errores se capturan e imprimen):

```
A = Matriz([[1, 2, 3], [4, 5, 6]])
B = Matriz([[7, 8], [9, 10], [11, 12]])
C = A * B
print(A.forma(), B.forma(), C.forma())
print(C)
print("Traza de C:", C.traza())
print("(AB)^T == B^T A^T:", C.transpuesta() == B.transpuesta() * A.transpuesta())

for intento in [lambda: A * A, lambda: A.traza(), lambda: Matriz([[1, 2], [3]])]:
    try:
        intento()
    except ValueError as e:
        print("Error:", e)
```

[14]: # Solución: